

Chapter 13:

Introduction to Classes

13.1

Procedural and Object-Oriented Programming

Procedural and Object-Oriented Programming

- Procedural programming focuses on the process/actions that occur in a program
- Object-Oriented programming is based on the data and the functions that operate on it. Objects are instances of ADTs (Abstract Data Types) that represent the data and its functions

Limitations of Procedural Programming

- If the data structures change, many functions must also be changed
- Programs that are based on complex function hierarchies are:
 - difficult to understand and maintain
 - difficult to modify and extend
 - easy to break

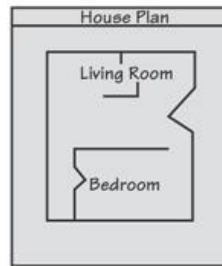
Object-Oriented Programming Terminology

- 🍊 class: like a `struct` (allows bundling of related variables), but variables and functions in the class can have different properties than in a `struct`
- 🍊 object: an instance of a `class`, in the same way that a variable can be an instance of a `struct`

Classes and Objects

- A Class is like a blueprint and objects are like houses built from the blueprint

Blueprint that describes a house.



Instances of the house described by the blueprint.



Object-Oriented Programming Terminology

- attributes: members of a class
- methods or behaviors: member functions of a class

More on Objects

- data hiding: restricting access to certain members of an object
- public interface: members of an object that are available outside of the object. This allows the object to provide access to some data and functions without sharing its internal details and design, and provides some protection from data corruption

13.2

Introduction to Classes

Introduction to Classes

- Objects are created from a `class`

- Format:

```
class ClassName
{
    declaration;
    declaration;
};
```

Class Example

```
class Rectangle
{
    private:
        double width;
        double length;
    public:
        void setWidth(double);
        void setLength(double);
        double getWidth() const;
        double getLength() const;
        double getArea() const;
};
```

Access Specifiers

- Used to control access to members of the class
- `public`: can be accessed by functions outside of the class
- `private`: can only be called by or accessed by functions that are members of the class

Class Example

```
class Rectangle
```

```
{
```

```
    private:
```

```
        double width;  
        double length;
```



Private Members

```
    public:
```

```
        void setWidth(double);  
        void setLength(double);  
        double getWidth() const;  
        double getLength() const;  
        double getArea() const;
```



Public Members

```
};
```

More on Access Specifiers

- 🍊 Can be listed in any order in a class
- 🍊 Can appear multiple times in a class
- 🍊 If not specified, the default is `private`

Using `const` With Member Functions

- 🍊 `const` appearing after the parentheses in a member function declaration specifies that the function will not change any data in the calling object.

```
double getWidth() const;  
double getLength() const;  
double getArea() const;
```

Defining a Member Function

- When defining a member function:
 - Put prototype in class declaration
 - Define function using class name and scope resolution operator (::)

```
int Rectangle::setWidth(double w)
{
    width = w;
}
```


Accessors and Mutators

- Mutator: a member function that stores a value in a private member variable, or changes its value in some way
- Accessor: function that retrieves a value from a private member variable.
Accessors do not change an object's data, so they should be marked `const`.

13.3

Defining an Instance of a Class

Defining an Instance of a Class

- An object is an instance of a class
- Defined like structure variables:

```
Rectangle r;
```

- Access members using dot operator:

```
r.setWidth(5.2);
```

```
cout << r.getWidth();
```

- Compiler error if attempt to access private member using dot operator

Program 13-1

```
1  // This program demonstrates a simple class.
2  #include <iostream>
3  using namespace std;
4
5  // Rectangle class declaration.
6  class Rectangle
7  {
8      private:
9          double width;
10         double length;
11     public:
12         void setWidth(double);
13         void setLength(double);
14         double getWidth() const;
15         double getLength() const;
16         double getArea() const;
17 };
18
19 //*****
20 // setWidth assigns a value to the width member.  *
21 //*****
22
23 void Rectangle::setWidth(double w)
24 {
25     width = w;
26 }
27
28 //*****
29 // setLength assigns a value to the length member. *
30 //*****
31
```

Program 13-1 (Continued)

```
32 void Rectangle::setLength(double len)
33 {
34     length = len;
35 }
36
37 //*****
38 // getWidth returns the value in the width member. *
39 //*****
40
41 double Rectangle::getWidth() const
42 {
43     return width;
44 }
45
46 //*****
47 // getLength returns the value in the length member. *
48 //*****
49
50 double Rectangle::getLength() const
51 {
52     return length;
53 }
54
```

Program 13-1 (Continued)

```
55  //*****
56  // getArea returns the product of width times length. *
57  //*****
58
59  double Rectangle::getArea() const
60  {
61      return width * length;
62  }
63
64  //*****
65  // Function main *
66  //*****
67
68  int main()
69  {
70      Rectangle box;      // Define an instance of the Rectangle class
71      double rectWidth;   // Local variable for width
72      double rectLength;  // Local variable for length
73
74      // Get the rectangle's width and length from the user.
75      cout << "This program will calculate the area of a\n";
76      cout << "rectangle. What is the width? ";
77      cin >> rectWidth;
78      cout << "What is the length? ";
79      cin >> rectLength;
80
81      // Store the width and length of the rectangle
82      // in the box object.
83      box.setWidth(rectWidth);
84      box.setLength(rectLength);
```

Program 13-1 (Continued)

```
85
86     // Display the rectangle's data.
87     cout << "Here is the rectangle's data:\n";
88     cout << "Width: " << box.getWidth() << endl;
89     cout << "Length: " << box.getLength() << endl;
90     cout << "Area: " << box.getArea() << endl;
91     return 0;
92 }
```

Program Output

This program will calculate the area of a rectangle. What is the width? **10 [Enter]**
What is the length? **5 [Enter]**
Here is the rectangle's data:
Width: 10
Length: 5
Area: 50

Avoiding Stale Data

- Some data is the result of a calculation.
- In the `Rectangle` class the area of a rectangle is calculated.
 - `length x width`
- If we were to use an `area` variable here in the `Rectangle` class, its value would be dependent on the `length` and the `width`.
- If we change `length` or `width` without updating `area`, then `area` would become *stale*.
- To avoid stale data, it is best to calculate the value of that data within a member function rather than store it in a variable.

Pointer to an Object

- Can define a pointer to an object:

```
Rectangle *rPtr = nullptr;
```

- Can access public members via pointer:

```
rPtr = &otherRectangle;
```

```
rPtr->setLength(12.5);
```

```
cout << rPtr->getLength() << endl;
```

Dynamically Allocating an Object

- We can also use a pointer to dynamically allocate an object.

```
1  // Define a Rectangle pointer.
2  Rectangle *rectPtr = nullptr;
3
4  // Dynamically allocate a Rectangle object.
5  rectPtr = new Rectangle;
6
7  // Store values in the object's width and length.
8  rectPtr->setWidth(10.0);
9  rectPtr->setLength(15.0);
10
11 // Delete the object from memory.
12 delete rectPtr;
13 rectPtr = nullptr;
```

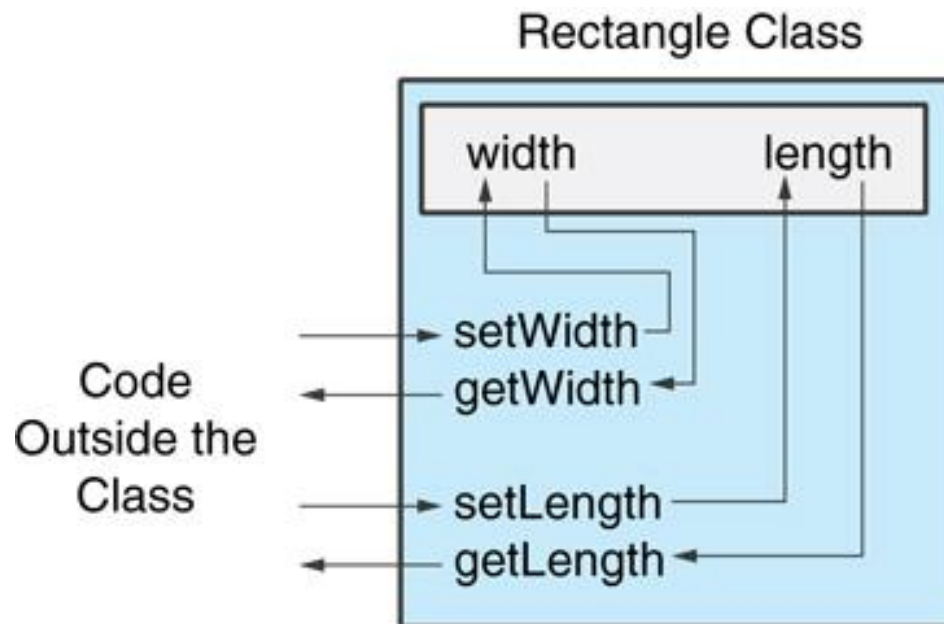
13.4

Why Have Private Members?

Why Have Private Members?

- Making data members `private` provides data protection
- Data can be accessed only through `public` functions
- Public functions define the class's public interface

Code outside the class must use the class's public member functions to interact with the object.



13.5

Separating Specification from
Implementation

Separating Specification from Implementation

- Place class declaration in a header file that serves as the class specification file. Name the file `ClassName.h`, for example, `Rectangle.h`
- Place member function definitions in `ClassName.cpp`, for example, `Rectangle.cpp` File should `#include` the class specification file
- Programs that use the class must `#include` the class specification file, and be compiled and linked with the member function definitions

13.6

Inline Member Functions

Inline Member Functions

- Member functions can be defined
 - inline: in class declaration
 - after the class declaration
- Inline appropriate for short function bodies:

```
int getWidth() const  
{ return width; }
```

Rectangle Class with Inline Member Functions

```
1  // Specification file for the Rectangle class
2  // This version uses some inline member functions.
3  #ifndef RECTANGLE_H
4  #define RECTANGLE_H
5
6  class Rectangle
7  {
8      private:
9          double width;
10         double length;
11     public:
12         void setWidth(double);
13         void setLength(double);
14
15         double getWidth() const
16             { return width; }
17
18         double getLength() const
19             { return length; }
20
21         double getArea() const
22             { return width * length; }
23     };
24 #endif
```

Tradeoffs – Inline vs. Regular Member Functions

- Regular functions – when called, compiler stores return address of call, allocates memory for local variables, etc.
- Code for an inline function is copied into program in place of call – larger executable program, but no function call overhead, hence faster execution

13.7

Constructors

Constructors

- Member function that is automatically called when an object is created
- Purpose is to construct an object
- Constructor function name is class name
- Has no return type

Contents of Rectangle.h (Version 3)

```
1 // Specification file for the Rectangle class
2 // This version has a constructor.
3 #ifndef RECTANGLE_H
4 #define RECTANGLE_H
5
6 class Rectangle
7 {
8     private:
9         double width;
10        double length;
11    public:
12        Rectangle();           // Constructor
13        void setWidth(double);
14        void setLength(double);
15
16        double getWidth() const
17            { return width; }
18
19        double getLength() const
20            { return length; }
21
22        double getArea() const
23            { return width * length; }
24 };
25 #endif
```

Contents of Rectangle.cpp (Version 3)

```
1  // Implementation file for the Rectangle class.
2  // This version has a constructor.
3  #include "Rectangle.h"    // Needed for the Rectangle class
4  #include <iostream>        // Needed for cout
5  #include <cstdlib>         // Needed for the exit function
6  using namespace std;
7
8  //*****
9  // The constructor initializes width and length to 0.0.      *
10 //*****
11
12 Rectangle::Rectangle()
13 {
14     width = 0.0;
15     length = 0.0;
16 }
```

Continues...

Contents of Rectangle.ccp Version3

```
17
18 //*****
19 // setWidth sets the value of the member variable width.    *
20 //*****
21
22 void Rectangle::setWidth(double w)
23 {
24     if (w >= 0)
25         width = w;
26     else
27     {
28         cout << "Invalid width\n";
29         exit(EXIT_FAILURE);
30     }
31 }
32
33 //*****
34 // setLength sets the value of the member variable length.  *
35 //*****
36
37 void Rectangle::setLength(double len)
38 {
39     if (len >= 0)
40         length = len;
41     else
42     {
43         cout << "Invalid length\n";
44         exit(EXIT_FAILURE);
45     }
46 }
```

(continued)

Program 13-7

```
1  // This program uses the Rectangle class's constructor.
2  #include <iostream>
3  #include "Rectangle.h" // Needed for Rectangle class
4  using namespace std;
5
6  int main()
7  {
8      Rectangle box;      // Define an instance of the Rectangle class
9
10     // Display the rectangle's data.
11     cout << "Here is the rectangle's data:\n";
12     cout << "Width: " << box.getWidth() << endl;
13     cout << "Length: " << box.getLength() << endl;
14     cout << "Area: " << box.getArea() << endl;
15     return 0;
16 }
```

Program Output

```
Here is the rectangle's data:
Width: 0
Length: 0
Area: 0
```

In-Place Initialization

- If you are using C++11 or later, you can initialize a member variable in its declaration statement, just as you can with a regular variable.
- This is known as in-place initialization. Here is an example:

```
class Rectangle
{
private:
    double width = 0.0;
    double length = 0.0;
public:
    Public member functions appear here...
};
```

Default Constructors

- A default constructor is a constructor that takes no arguments.
- If you write a class with no constructor at all, C++ will write a default constructor for you, one that does nothing.
- A simple instantiation of a class (with no arguments) calls the default constructor:

```
Rectangle r;
```

13.8

Passing Arguments to Constructors

Passing Arguments to Constructors

- To create a constructor that takes arguments:

- indicate parameters in prototype:

```
Rectangle(double, double);
```

- Use parameters in the definition:

```
Rectangle::Rectangle(double w, double len)
{
    width = w;
    length = len;
}
```

Passing Arguments to Constructors

- You can pass arguments to the constructor when you create an object:

```
Rectangle r(10, 5);
```

More About Default Constructors

- If all of a constructor's parameters have default arguments, then it is a default constructor. For example:

```
Rectangle(double = 0, double = 0);
```

- Creating an object and passing no arguments will cause this constructor to execute:

```
Rectangle r;
```

Classes with No Default Constructor

- 🍊 When all of a class's constructors require arguments, then the class has NO default constructor.
- 🍊 When this is the case, you must pass the required arguments to the constructor when creating an object.

13.9

Destructors

Destructors

- Member function automatically called when an object is destroyed
- Destructor name is ~classname, *e.g.*,
~Rectangle
- Has no return type; takes no arguments
- Only one destructor per class, *i.e.*, it cannot be overloaded
- If constructor allocates dynamic memory, destructor should release it

Contents of InventoryItem.h (Version 1)

```
1  // Specification file for the InventoryItem class.
2  #ifndef INVENTORYITEM_H
3  #define INVENTORYITEM_H
4  #include <cstring>    // Needed for strlen and strcpy
5
6  // InventoryItem class declaration.
7  class InventoryItem
8  {
9  private:
10     char *description; // The item description
11     double cost;        // The item cost
12     int units;          // Number of units on hand
```

Contents of InventoryItem.h Version1

```
13 public:
14     // Constructor
15     InventoryItem(char *desc, double c, int u)
16     { // Allocate just enough memory for the description.
17         description = new char [strlen(desc) + 1];
18
19         // Copy the description to the allocated memory.
20         strcpy(description, desc);
21
22         // Assign values to cost and units.
23         cost = c;
24         units = u;}
25
26     // Destructor
27     ~InventoryItem()
28     { delete [] description; }
29
30     const char *getDescription() const
31     { return description; }
32
33     double getCost() const
34     { return cost; }
35
36     int getUnits() const
37     { return units; }
38 };
39 #endif
```

(continued)

Program 13-12

```
1  // This program demonstrates a class with a destructor.
2  #include <iostream>
3  #include "ContactInfo.h"
4  using namespace std;
5
6  int main()
7  {
8      // Define a ContactInfo object with the following data:
9      // Name: Kristen Lee Phone Number: 555-2021
10     ContactInfo entry("Kristen Lee", "555-2021");
11
12     // Display the object's data.
13     cout << "Name: " << entry.getName() << endl;
14     cout << "Phone Number: " << entry.getPhoneNumber() << endl;
15     return 0;
16 }
```

Program Output

Name: Kristen Lee
Phone Number: 555-2021

Constructors, Destructors, and Dynamically Allocated Objects

- When an object is dynamically allocated with the `new` operator, its constructor executes:

```
Rectangle *r = new Rectangle(10, 20);
```

- When the object is destroyed, its destructor executes:

```
delete r;
```

13.10

Overloading Constructors

Overloading Constructors

- A class can have more than one constructor
- Overloaded constructors in a class must have different parameter lists:

```
Rectangle();
```

```
Rectangle(double);
```

```
Rectangle(double, double);
```



```

1  // This class has overloaded constructors.
2  #ifndef INVENTORYITEM_H
3  #define INVENTORYITEM_H
4  #include <string>
5  using namespace std;
6
7  class InventoryItem
8  {
9  private:
10     string description; // The item description
11     double cost;        // The item cost
12     int units;          // Number of units on hand
13 public:
14     // Constructor #1
15     InventoryItem()
16     { // Initialize description, cost, and units.
17         description = "";
18         cost = 0.0;
19         units = 0; }
20
21     // Constructor #2
22     InventoryItem(string desc)
23     { // Assign the value to description.
24         description = desc;
25
26         // Initialize cost and units.
27         cost = 0.0;
28         units = 0; }

```

Continues...

```
29
30 // Constructor #3
31 InventoryItem(string desc, double c, int u)
32 { // Assign values to description, cost, and units.
33     description = desc;
34     cost = c;
35     units = u; }
36
37 // Mutator functions
38 void setDescription(string d)
39     { description = d; }
40
41 void setCost(double c)
42     { cost = c; }
43
44 void setUnits(int u)
45     { units = u; }
46
47 // Accessor functions
48 string getDescription() const
49     { return description; }
50
51 double getCost() const
52     { return cost; }
53
54 int getUnits() const
55     { return units; }
56 };
57 #endif
```

Constructor Delegation

- Sometimes a class will have multiple constructors that perform a similar set of steps. For example, look at the following Contact class:

```
class Contact
{
private:
    string name;
    string email;
    string phone;
public:
    // Constructor #1 (default)
    Contact()
    { name = "";
      email = "";
      phone = "";
    }

    // Constructor #2
    Contact(string n, string e, string p)
    { name = n;
      email = e;
      phone = p;
    }

    Other member functions follow...
};
```

Constructor Delegation

- Both constructors perform a similar operation: They assign values to the name, email, and phone member variables.
- The default constructor assigns empty strings to the members, and the parameterized constructor assigns specified values to the members.

```
class Contact
{
private:
    string name;
    string email;
    string phone;
public:
    // Constructor #1 (default)
    Contact()
    { name = "";
      email = "";
      phone = "";
    }

    // Constructor #2
    Contact(string n, string e, string p)
    { name = n;
      email = e;
      phone = p;
    }

    Other member functions follow...
};
```

Constructor Delegation

- In C++ 11, it is possible for one constructor to call another constructor in the same class.
- This is known as *constructor delegation*.

```
class Contact
{
private:
    string name;
    string email;
    string phone;
public:
    // Constructor #1 (default)
    Contact() : Contact("", "", "")
    { }

    // Constructor #2
    Contact(string n, string e, string p)
    { name = n;
      email = e;
      phone = p;
    }

    Other member functions follow...
};
```

Only One Default Constructor and One Destructor

- Do not provide more than one default constructor for a class: one that takes no arguments and one that has default arguments for all parameters

```
Square();
```

```
Square(int = 0); // will not compile
```

- Since a destructor takes no arguments, there can only be one destructor for a class

Member Function Overloading

- Non-constructor member functions can also be overloaded:

```
void setCost(double) ;
```

```
void setCost(char *) ;
```

- Must have unique parameter lists as for constructors

3.11

Using Private Member Functions

Using Private Member Functions

- A `private` member function can only be called by another member function
- It is used for internal processing by the class, not for use outside of the class
- See the `createDescription` function in **`ContactInfo.h`** (Version 2)

13.12

Arrays of Objects

Arrays of Objects

- Objects can be the elements of an array:

```
InventoryItem inventory[40];
```

- Default constructor for object is used when array is defined

Arrays of Objects

- Must use initializer list to invoke constructor that takes arguments:

```
InventoryItem inventory[3] =  
    { "Hammer", "Wrench", "Pliers" };
```

Arrays of Objects

- 🍊 If the constructor requires more than one argument, the initializer must take the form of a function call:

[illegible]

Arrays of Objects

- It isn't necessary to call the same constructor for each object in an array:

```
InventoryItem inventory[3] = { "Hammer",  
                               InventoryItem("Wrench", 8.75, 20),  
                               "Pliers" };
```

Accessing Objects in an Array

- Objects in an array are referenced using subscripts
- Member functions are referenced using dot notation:

```
inventory[2].setUnits(30);  
cout << inventory[2].getUnits();
```

Program 13-14

```
1  // This program demonstrates an array of class objects.
2  #include <iostream>
3  #include <iomanip>
4  #include "InventoryItem.h"
5  using namespace std;
6
7  int main()
8  {
9      const int NUM_ITEMS = 5;
10     InventoryItem inventory[NUM_ITEMS] = {
11         InventoryItem("Hammer", 6.95, 12),
12         InventoryItem("Wrench", 8.75, 20),
13         InventoryItem("Pliers", 3.75, 10),
14         InventoryItem("Ratchet", 7.95, 14),
15         InventoryItem("Screwdriver", 2.50, 22) };
16
```


Program 13-14 (Continued)

```
17     cout << setw(14) << "Inventory Item"
18         << setw(8) << "Cost" << setw(8)
19         << setw(16) << "Units on Hand\n";
20     cout << "-----\n";
21
22     for (int i = 0; i < NUM_ITEMS; i++)
23     {
24         cout << setw(14) << inventory[i].getDescription();
25         cout << setw(8) << inventory[i].getCost();
26         cout << setw(7) << inventory[i].getUnits() << endl;
27     }
28
29     return 0;
30 }
```

Program Output

Inventory Item	Cost	Units on Hand
Hammer	6.95	12
Wrench	8.75	20
Pliers	3.75	10
Ratchet	7.95	14
Screwdriver	2.5	22

13.16

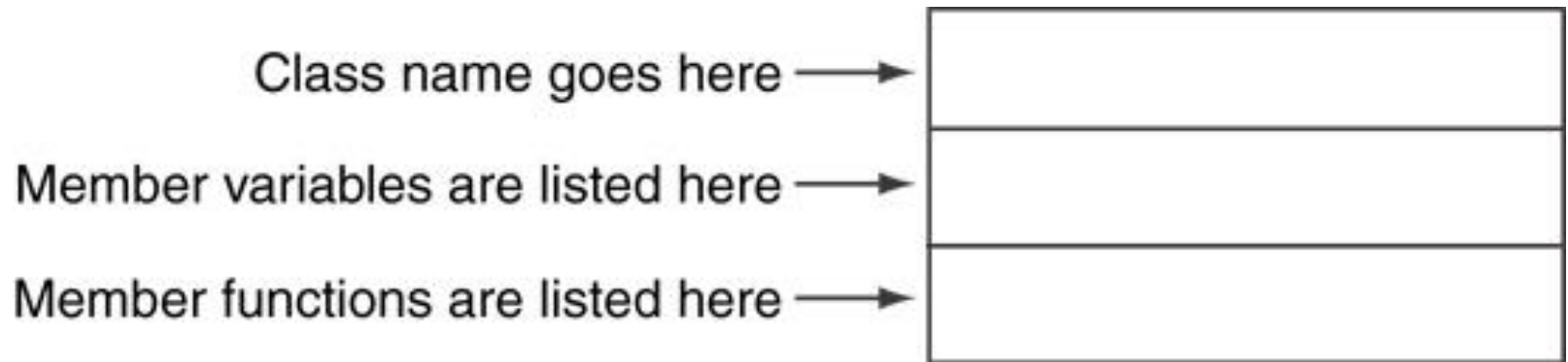
The Unified Modeling Language

The Unified Modeling Language

- *UML* stands for *Unified Modeling Language*.
- The UML provides a set of standard diagrams for graphically depicting object-oriented systems

UML Class Diagram

- A UML diagram for a class has three main sections.



Example: A Rectangle Class

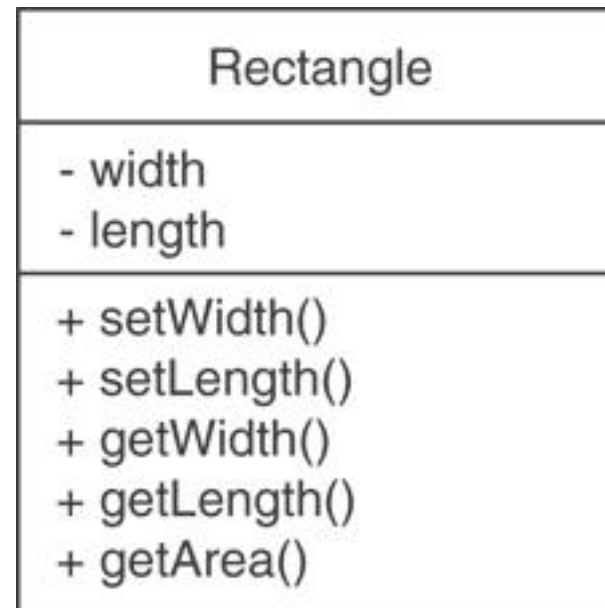
Rectangle
width length
setWidth() setLength() getWidth() getLength() getArea()

```
class Rectangle
{
    private:
        double width;
        double length;
    public:
        bool setWidth(double);
        bool setLength(double);
        double getWidth() const;
        double getLength() const;
        double getArea() const;
};
```

UML Access Specification Notation

- In UML you indicate a private member with a minus (-) and a public member with a plus(+).

These member variables are private.



These member functions are public.



UML Data Type Notation

- To indicate the data type of a member variable, place a colon followed by the name of the data type after the name of the variable.
 - width : double
 - length : double

UML Parameter Type Notation

- To indicate the data type of a function's parameter variable, place a colon followed by the name of the data type after the name of the variable.

+ setWidth(w : double)

UML Function Return Type Notation

- To indicate the data type of a function's return value, place a colon followed by the name of the data type after the function's parameter list.

```
+ setWidth(w : double) : void
```

The Rectangle Class

Rectangle
- width : double - length : double
+ setWidth(w : double) : bool + setLength(len : double) : bool + getWidth() : double + getLength() : double + getArea() : double

Showing Constructors and Destructors

No return type listed for constructors or destructors

Constructors →

Destructor →

InventoryItem	
- description : char* - cost : double - units : int - createDescription(size : int, value : char*) : void	
	+ InventoryItem() : + InventoryItem(desc : char*) : + InventoryItem(desc : char*, c : double, u : int) : + ~InventoryItem() : + setDescription(d : char*) : void + setCost(c : double) : void + setUnits(u : int) : void + getDescription() : char* + getCost() : double + getUnits() : int